AI-Driven Testing and Code Review: Quality Assurance in the Agentic Era

Muhammad Swalha and Mohammed Abad

Course: Mass Production of AI Agents
Lecturer: Dr. Yoram Segal

2026-06-12

Contents

1 Introduction — The Dawn of a New QA Paradigm

Software quality assurance has never stood still. From the earliest days of manual testing — where dedicated testers painstakingly executed test scripts by hand — to the rise of automated frameworks capable of replaying hundreds of test cases in minutes, QA has repeatedly reinvented itself in response to the demands of software development. Yet none of these prior reinventions quite prepared the industry for the transformation now underway. Artificial intelligence, and specifically the emergence of agentic AI, is reshaping quality assurance at a speed and scale that demands both excitement and careful scrutiny. The industry is not witnessing incremental improvement but a categorical shift in what it means to test software and review code.

The drivers of this shift are interconnected and mutually reinforcing. AI-assisted development tools — from GitHub Copilot to a growing constellation of large language model (LLM) integrations — have dramatically accelerated the rate at which developers produce code. What once took weeks can now take hours; what took hours can take minutes. This acceleration is a double-edged sword: while it empowers development teams to move faster, it simultaneously compresses the time available for quality assurance and increases the sheer volume of code that must be tested, reviewed, and validated. Traditional QA processes, already struggling to keep pace with agile and DevOps delivery cadences, now face an existential challenge [19]. The question is not simply whether AI can help with testing but whether any approach other than AI-driven QA is viable in the current development landscape.

At the same time, the economic signals are unmistakable. The agentic AI market — encompassing AI systems capable of autonomous, goal-directed action — is projected to grow from USD 7.06 billion in 2025 to USD 93.20 billion by 2032, representing a compound annual growth rate of 44.6% [4]. This explosive growth reflects adoption across industries, but software quality assurance is among the domains where the returns are most immediate and most tangible. Organizations that integrate agentic AI into their QA workflows report shorter release cycles, fewer production defects, and meaningful reductions in the manual labor burden on testing teams. These are not speculative benefits; they are measurable outcomes already appearing in production environments worldwide.

The projected market trajectory can be expressed precisely. If the market value in year t grows at a constant compound annual rate r, then its value after n years is:

$$V(n) = V_0 \cdot (1 + r)^n \tag{1}$$

where $V_0 = 7.06$ billion USD, $r = 0.446$, and $n = 7$ (from 2025 to 2032), yielding the projected value of approximately USD 93.20 billion.

This article examines the full landscape of AI-driven testing and code review in the agentic era. It covers the conceptual foundations of agentic QA, the technical capabilities of autonomous test generation and self-healing automation, the evolving practice of AI-assisted code review, the genuine risks and challenges that must be navigated, and the human-oversight frameworks that remain essential even as AI takes on more of the work. It concludes by sketching the future of quality assurance: not a world without human testers, but one where human testers are liberated from toil and empowered to focus on judgment, strategy, and accountability. Throughout, the goal is to provide practitioners, leaders, and researchers with a clear-eyed, evidence-based account of where QA stands today and where it is headed.

2 Understanding Agentic AI in Quality Assurance

The term ”agentic AI” is used broadly in contemporary discourse, but in the context of quality assurance it carries a specific and important meaning. Agentic AI refers to AI systems that can autonomously perceive their environment, formulate plans to achieve defined goals, take actions to execute those plans, and iterate on their approach based on feedback — all with minimal human intervention between steps [7]. This is fundamentally different from rule-based automation, which executes predetermined scripts without adaptation, and from earlier generations of AI-assisted QA tools, which provided recommendations that humans then acted upon. Agentic systems act. They are not advisors; they are active participants in the QA process, capable of multi-step reasoning and problem-solving in ways that begin to mirror the cognitive processes of experienced human testers.

To understand why this distinction matters, consider the traditional test automation lifecycle. A QA engineer writes a test script — often in Selenium or Cypress — that clicks through a series of UI elements in a specific sequence. When the application changes (a button moves, a form field is renamed, an API endpoint is modified), the script breaks. The engineer must diagnose the failure, identify which element selector no longer matches, update the script, and re-run the test. This cycle consumes enormous QA bandwidth, often called ”test maintenance hell” in the industry. Agentic AI fundamentally disrupts this cycle. Rather than executing a rigid script, an agentic system perceives the current state of the application, reasons about the intended test goal, and determines how to achieve that goal given the present UI or API structure [13]. When the application changes, the agent adapts rather than breaks.

The distinction between agentic AI and conventional automation also matters at the strategic level. Traditional automation tools require QA teams to know in advance what to test and to express that knowledge in formal scripts. Agentic AI can analyze an application dynamically, assess its risk profile, identify which components are most likely to contain defects, and construct a testing strategy accordingly — all without requiring a human to specify every test case [6]. This shifts the role of QA professionals from script writers to strategy definers and oversight providers — a significant cognitive and organizational transition that requires not just new tools but new ways of thinking about what QA teams are for.

The momentum behind agentic QA is visible in adoption data. Stack Overflow’s 2024 Developer Survey found that 63% of professional developers are currently using AI in their development process, with an additional 14% planning to adopt AI tools in the near future [9]. This near-universal integration of AI into development workflows creates immediate pressure on QA to match pace. If developers generate code faster with AI assistance, the testing and review pipeline must be able to absorb and validate that code at comparable speed. Agentic QA is the response the industry is converging on — driven not by enthusiasm for novelty but by the practical reality that the alternative (scaling manual testing teams proportionally to code-volume growth) is neither economically sustainable nor organizationally feasible.

Looking ahead, 2026 is shaping up to be a pivotal year for agentic QA adoption. Industry analysts and practitioners anticipate that advanced agentic testing will move from experimental deployments to mainstream production use, that new workforce-training standards will emerge to prepare QA professionals for agentic collaboration, and that the fundamental definition of QA roles will be reshaped by the capabilities that agentic

systems bring to the table [17]. Organizations that navigate this transition thoughtfully — investing in both the technology and the human capabilities needed to work alongside it — will emerge with a durable competitive advantage in software quality.

3 Autonomous Test Generation and Intelligent Test Strategy

One of the most transformative capabilities that agentic AI brings to QA is the ability to generate test cases automatically, drawing on requirements documents, user stories, application code, and behavioral patterns observed in running systems. This capability strikes at one of the most labor-intensive aspects of the traditional QA lifecycle: creating a comprehensive test suite. In conventional environments, test-case authoring is a skilled and time-consuming activity that demands both domain knowledge and technical expertise. A test engineer must understand what the software is supposed to do, translate that understanding into executable test scenarios, and then maintain those scenarios as the software evolves. Agentic AI can perform all of these steps autonomously, and it can do so continuously rather than in periodic bursts aligned with release cycles [6].

The mechanism behind agentic test generation combines the reasoning capabilities of large language models with structured access to application artifacts. Given a user story describing a checkout flow in an e-commerce application, for instance, an agentic system can parse the acceptance criteria, infer the boundary conditions and edge cases that a thorough tester would consider, generate test cases covering both happy paths and failure modes, and express those cases in a format compatible with the organization's existing test-execution infrastructure. More advanced implementations can also analyze the application's code directly, identifying branches, conditionals, and data transformations that require explicit test coverage — approaching what experienced engineers call "white-box testing," but executed at a scale and speed no human team could sustain alone.

Autonomous test strategy creation represents an even more sophisticated application of agentic capabilities. Rather than generating individual test cases, these systems assess the overall risk profile of an application or a code change and construct a comprehensive testing strategy that allocates coverage proportionally to risk [11]. This mirrors the risk-based testing methodologies that senior QA engineers have long championed but rarely had the bandwidth to implement rigorously. An agentic system with access to defect history, code-complexity metrics, change-frequency data, and business-impact assessments can make principled decisions about where to invest testing effort — prioritizing high-risk components without neglecting low-risk ones. The result is a testing strategy that is both more intelligent and more efficiently executed than what most human teams can deliver under time pressure.

The benefits of autonomous test generation extend beyond immediate efficiency gains. Organizations using agentic AI testing have reported reduced manual test-maintenance costs, shorter release cycles, and fewer production defects [5]. Furthermore, AI-generated test suites tend to provide more comprehensive edge-case coverage than human-authored suites, because human testers naturally gravitate toward intuitive scenarios while AI systems apply consistent criteria across the entire application behavior space. This does not mean AI-generated tests are perfect; it means they introduce a different and often complementary coverage pattern that, combined with human judgment, produces better outcomes than either approach alone.

A particularly significant development is the democratization of test creation through plain-English authoring enabled by agentic AI. Traditional automation required QA engineers to write code — a prerequisite that excluded business analysts, product owners, and other non-technical stakeholders from directly participating in test creation [@perfecto2025agenticqa; @virtuoso2025redefining]. Agentic systems that accept natural-language descriptions of desired behavior and translate them into executable tests lower this barrier dramatically. When a product owner can describe a test scenario in plain English and have it executed automatically, the boundary between specification and verification blurs in productive ways. Requirements become tests, and the act of defining what the software should do simultaneously creates the means of verifying that it does so — an alignment that quality-focused development has long sought and that agentic AI is now making practically achievable at scale.

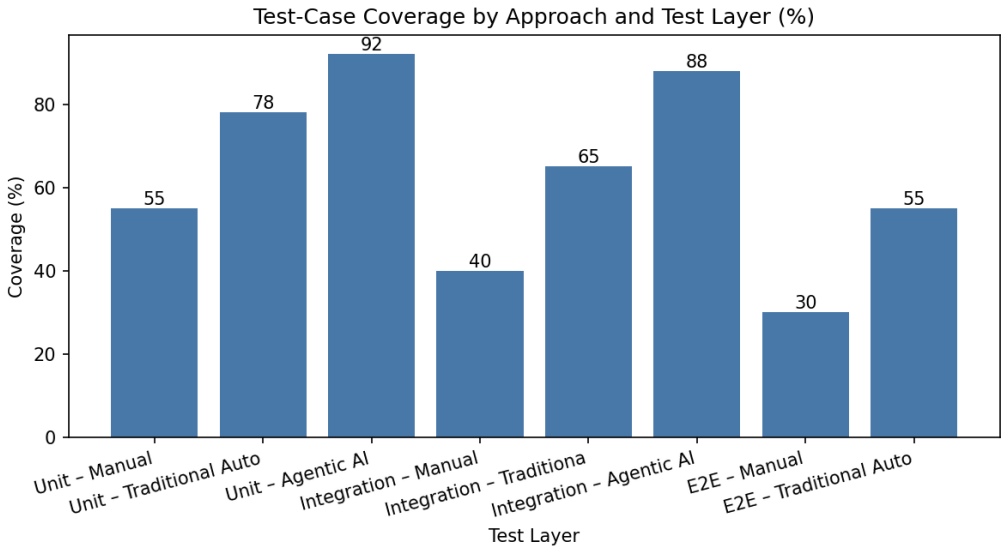

Figure 1: Bar chart comparing test-case coverage percentages achieved by manual testing, traditional automation, and agentic AI-driven test generation across unit, integration, and end-to-end test layers

4 Self-Healing Test Automation and CI/CD Integration

The fragility of automated test suites has long been one of the most persistent frustrations in software QA. Test maintenance — updating test scripts when application changes break them — consumes a disproportionate share of QA engineering time and often undermines the ROI justification for automation investment. In large enterprises, a substantial fraction of QA team effort goes not to creating new tests but to fixing existing ones. Agentic AI addresses this problem directly through what the industry calls self-healing test automation: the ability of an AI system to detect when a test has failed due to an application change rather than a genuine defect, diagnose the specific cause, and repair the test automatically so that it continues to function as intended [1].

Self-healing works by giving the AI agent a rich model of the application's structure and the test's intent. When a test fails, the agent investigates rather than simply report-

ing the failure. It compares the current state of the application with the expected state, identifies discrepancies — a changed element selector, a modified API response structure, a renamed database field — and determines whether those discrepancies represent a maintenance issue or a genuine defect. If the failure is attributable to a legitimate application change, the agent updates the test to accommodate the new structure and re-runs it. If the failure indicates a real bug, the agent escalates appropriately, flagging the issue for human review with contextual diagnostic information attached [2]. This two-tier diagnostic process eliminates a large category of false positives and ensures that human attention is directed toward genuine quality issues rather than routine maintenance tasks.

Integrating agentic test execution with continuous integration and continuous delivery (CI/CD) pipelines represents another major leap forward. In a mature CI/CD environment, code is committed dozens or hundreds of times per day, and each commit ideally triggers a comprehensive suite of automated checks. Agentic AI plugs directly into this pipeline — tests run after every commit, results are automatically analyzed, and insights flow to developers in real time without human orchestration in the middle [2]. The agent does not simply execute tests and report pass/fail; it interprets patterns across multiple runs, identifies regressions introduced by specific commits, correlates failures with code changes, and surfaces root-cause analysis in actionable form. The feedback loop becomes so tight that developers can address quality issues while the relevant code is still fresh in their minds.

Root-cause detection deserves particular emphasis because it represents a qualitative advance over earlier generations of test automation. Traditional automated testing told teams *that* something was broken; it rarely explained *why*. Developers receiving a failure report had to investigate on their own, often spending more time diagnosing the failure than they spent on the original code change. Agentic systems that trace a test failure back to its origin — identifying not just the failing test but the specific commit, the specific line, and often the specific logical error — transform failure reports from opaque work items into near-direct guidance [@azirotech2025selfhealing; @zyrix2025revolution]. This capability dramatically accelerates defect resolution and reduces the cognitive overhead that makes failure investigation so frustrating in traditional environments.

For organizations considering adoption, self-healing automation and CI/CD integration are not optional enhancements — they are foundational capabilities that determine whether promised benefits materialize in practice. An agentic test-generation system that produces excellent tests which then break on every deployment and require manual repair has not solved the test-maintenance problem; it has merely moved it. The full value of agentic QA is realized when test generation, execution, self-healing, and pipeline integration work in concert, creating a system that is not just automated but genuinely autonomous — capable of sustaining its own effectiveness without continuous human intervention [3].

5 AI-Driven Code Review — From Pull Requests to Agentic Workflows

Code review is one of the most valuable practices in software engineering, consistently shown to catch defects earlier and more cheaply than almost any other quality mechanism. Yet traditional code review is also one of the most expensive practices in terms of senior engineer time. A thorough review requires understanding the context of a change,

evaluating its correctness, assessing its security implications, considering its performance characteristics, verifying adherence to coding standards, and providing constructive feedback to the author — all while managing one's own development responsibilities. In busy teams, code review becomes a bottleneck: pull requests queue up, review quality degrades under time pressure, and the practice meant to improve quality can degrade into rubber-stamping that adds delay without adding value.

AI-driven code review tools address this bottleneck by automating the most repetitive and systematic aspects of the review process. These tools scan automatically for bugs, style violations, security vulnerabilities, and performance issues before code reaches production, handling the categories of checks that human reviewers find tedious and are most likely to miss when attention is divided [@mend2025aireview; @digitalapplied2025guide]. The AI code review market reflects eagerness to adopt these capabilities: valued at USD 6.7 billion in 2024, with AI now contributing to roughly 41% of new code, AI code review is not a supplementary tool but a central pillar of modern software quality practice [8]. When AI generates code at scale, AI review of that code is not just convenient — it is essential for maintaining any coherent quality standard.

GitHub Copilot's code review capabilities represent the most visible entry point for most development organizations. Now generally available, Copilot's pull-request automation features include agentic workflows and line-level feedback that surface issues directly in the pull request interface — the environment where developers already work [12]. This integration strategy is significant: by meeting developers where they are, rather than requiring adoption of a separate tool, AI code review achieves adoption rates that dedicated external tools struggle to match. The line-level feedback model also aligns with how experienced human reviewers communicate — pointing to specific lines and explaining exactly what the issue is and why it matters, rather than providing vague general assessments. Developers consistently report that this granularity makes AI code review feedback actionable in ways that higher-level automated reports often are not.

The 2026 ICSE research paper "Toward Agentic Code Review: Reimagining the Process in the AI Era" provides important academic grounding for where the field is heading [20]. The paper revisits the historical trajectory of code review practice, identifies persistent gaps in efficiency and traceability that conventional review processes have never fully resolved, and proposes an agentic framework for next-generation automated code review. The framework envisions code review not as a checklist-driven quality gate but as an active reasoning process in which AI agents understand the intent of a change, evaluate it against the broader codebase context, identify interactions and dependencies a human reviewer might miss, and generate feedback that goes beyond style and syntax to address architectural coherence and behavioral correctness.

The practical impact of AI code review on development team dynamics is multifaceted. On the positive side, AI review democratizes access to expert-level feedback: a junior developer without access to a senior reviewer in the same time zone can still receive immediate, detailed feedback from an AI system trained on vast corpora of high-quality code. On the challenging side, the volume of AI-generated feedback can itself become burdensome if not carefully managed. Teams report that without proper calibration, AI code review tools generate so many comments — many valid but low-priority — that developers begin ignoring the feedback out of fatigue, mirroring the alert-fatigue problem well known in security and monitoring contexts [10]. Managing the signal-to-noise ratio is not a secondary concern; it is one of the primary design challenges separating effective implementations from ineffective ones.

6 Challenges, Risks, and the Hallucination Problem

The promise of agentic AI in QA is substantial, but the technology carries genuine risks that practitioners must understand clearly. Among the most technically significant is the phenomenon of hallucination in large language model outputs. In the context of AI code review and test generation, hallucination refers to cases where the AI system produces output that is confident and syntactically coherent but factually incorrect — identifying a bug that does not exist, recommending a fix that introduces a new vulnerability, or generating a test case that does not actually validate the behavior it claims to assess. Research through 2024 and 2025 has made meaningful progress: mitigation strategies have been identified that can reduce LLM hallucinations in code review by up to 96% in controlled settings [14]. However, no current tool eliminates hallucinations entirely, and even a small residual rate can represent a significant absolute number of erroneous outputs in high-volume deployments.

The false-positive problem — AI code review tools flagging issues that are not actually issues — received systematic academic attention in a 2025 IACIS study examining developer perceptions of AI-assisted code review [16]. Developers reported that AI tools sometimes generate false positives at a rate that partially offsets their time-saving benefits, because investigating and dismissing false positives consumes time and cognitive effort. More troubling, the study found that when developers learn to expect false positives from a tool, they begin to discount its output more broadly, potentially missing genuine issues that are correctly identified. This calibration problem — establishing the right level of trust in AI outputs — is particularly acute in code review because the consequences of both missed defects and unwarranted changes can be severe.

Beyond technical risks, there are organizational and strategic risks associated with transitioning to agentic QA. The central risk is over-reliance: teams that delegate too much to AI agents without maintaining the human expertise and oversight needed to validate AI outputs may find that they have optimized for speed at the expense of genuine quality. This risk is not hypothetical — there are already documented cases of organizations deploying AI coding tools without adequate QA adaptation, only to discover that systemic interactions between AI-generated components introduced subtle defects that only manifested in production [15]. The speed that agentic AI enables is only valuable when paired with the wisdom to know what the AI cannot yet do reliably.

Signal-to-noise has emerged as the dominant QA challenge of 2026, and it operates differently from traditional QA bottlenecks [10]. In traditional environments, the bottleneck was usually execution speed — more tests to run than time available. In agentic environments, execution is cheap and fast, but the volume of output — test results, AI code review comments, automated analysis reports — can overwhelm the human capacity to interpret and act on it meaningfully. When developers and QA engineers receive hundreds of automated findings per day, the cognitive load of triage becomes prohibitive, and the risk is not that issues go undetected but that they go unaddressed amid a flood of lower-priority notifications. Solving this problem requires better AI governance: clear policies about which findings must be addressed before code advances, which can be deferred, and which can be suppressed based on team-specific criteria.

The question of accountability and trust in agentic QA systems is perhaps the most profound organizational challenge. In a traditional QA environment, accountability is clear: a specific person signed off on a test plan, executed a test run, and approved a release. In an agentic environment where AI agents generate tests, execute them, interpret

results, and potentially approve code advances autonomously, the chain of accountability becomes diffuse. A central debate in the industry concerns whether AI agents should assist testers or autonomously execute and approve test cases — a question of trust, accountability, and co-ownership of QA cycles that the industry has not yet resolved with consensus [18]. Regulatory environments, particularly in healthcare, finance, and aviation, are beginning to ask pointed questions about how organizations can demonstrate the validity and accountability of AI-mediated quality processes, underscoring the importance of treating human oversight not as a temporary concession to AI immaturity but as a permanent architectural feature of responsible agentic QA systems.

7 Comparison of QA Approaches

The following table summarizes the key characteristics of manual testing, traditional automation, and agentic AI-driven QA across the dimensions most relevant to modern software delivery:

Dimension	Manual Testing	Traditional Automation	Agentic AI QA
Test creation	Fully human-authored	Human-scripted	AI-generated from requirements/code
Adaptation to change	Human re-evaluates	Script breaks; manual fix required	Self-healing; agent adapts automatically
Execution speed	Slow; sequential	Fast; parallel	Fast; parallel and continuous
Coverage breadth	Limited by time and bandwidth	Limited to scripted scenarios	Comprehensive, risk-prioritized
Edge-case discovery	Dependent on tester experience	Dependent on script author	Systematic; AI applies consistent criteria
False-positive risk	Low (human judgment)	Medium (brittle selectors)	Requires calibration; hallucination risk
CI/CD integration	Manual gate	Integrated but rigid	Natively integrated; autonomous feedback
Human oversight required	Full	Partial (script writing, triage)	Strategic (governance, validation, policy)
Scalability	Poor	Moderate	High
Key challenge	Speed and volume	Maintenance cost	Signal-to-noise and accountability

8 Human Oversight in the Age of Autonomous QA

The discourse around AI in QA has sometimes been framed as a choice between human testing and AI testing, as if the two were mutually exclusive alternatives competing for the same organizational resources. This framing is not just inaccurate; it is counterproductive, leading organizations to either under-adopt AI tools out of protectionism or over-adopt them out of automation enthusiasm — without developing the thoughtful integration model that actually produces superior quality outcomes. The evidence from organizations that have implemented agentic QA at scale is consistent: AI handles scale, speed, and pattern recognition; humans provide judgment, context, and accountability [@perfecto2025agenticqa; @allis2025saasriseqa]. Neither can substitute for the other.

Human oversight in agentic QA is not simply a safety net for AI errors, though it certainly serves that function. It is also the source of the contextual intelligence that AI systems cannot yet generate independently. An AI system can determine with high confidence that a piece of code contains a SQL injection vulnerability; it takes human judgment to evaluate whether that vulnerability is exploitable in the actual deployment context, whether compensating controls are in place, and what the business risk tolerance is for accepting technical debt against a future remediation commitment. Similarly, an AI system can generate a test case for every branch of a complex algorithm; it takes a human domain expert to evaluate whether those test cases collectively represent the real-world usage patterns that matter most for the organization's users [18].

The redefinition of QA roles in the agentic era is both an opportunity and a challenge. Up to 90% of QA professionals are expected to advance to strategic roles through AI collaboration — shifting from manual test execution to higher-order work such as risk assessment, tooling governance, and AI supervision [11]. In principle, this is a positive development: manual test-execution tasks are among the less intellectually engaging aspects of the role, and freeing capacity for strategic and analytical work should increase both team effectiveness and job satisfaction. In practice, however, this transition is not automatic. QA professionals who have spent their careers in manual execution or script-based automation need to develop new competencies — in prompt engineering, AI output validation, risk-based test strategy, and tooling governance — to thrive alongside agentic tools.

GitHub's position on AI-assisted code review captures an important nuance: agentic AI reshaping QA and code review does not eliminate the need for fundamentals. Code review, verification, and accountability remain as critical as ever — AI changes how these activities are performed, not whether they are performed [12]. The existence of AI code review tools does not relieve engineering leads of the responsibility to ensure that code review is happening, that it is substantive rather than performative, and that the organization has the data needed to demonstrate the effectiveness of its quality processes.

Constructing effective human-AI teaming arrangements requires attention to the specifics of handoff design: which decisions AI agents make autonomously, which require human confirmation before action, and which must always be made by humans regardless of AI input. Mature implementations establish these handoff rules explicitly rather than leaving them ambiguous, and build monitoring mechanisms that can detect when AI agents are operating outside their reliable performance envelope. In practice, this means QA organizations need AI governance skills — the ability to define policies, monitor compliance, audit outcomes, and adjust the autonomy level of AI agents as their reliability is validated over time [10].

9 Best Practices for Agentic QA Implementation

Implementing agentic QA is not a plug-and-play exercise. Organizations that approach it as simply procuring a tool and pointing it at their codebase are consistently disappointed with the results. Those that approach it as an organizational transformation — requiring thoughtful planning, stakeholder alignment, technical integration, and ongoing governance — consistently report the benefits that the technology's proponents describe. Effective agentic QA requires intentional tooling choices, trust-building in AI outputs, and clear governance over which decisions AI agents make autonomously versus which require human sign-off [10].

The starting point for any agentic QA implementation should be an honest assessment of the organization's current QA maturity. Teams with strong existing automation foundations, clear coverage metrics, and established defect-tracking practices are well positioned to extend those foundations with agentic capabilities. Teams with fragmented, largely manual QA practices that lack systematic coverage or metrics will find that agentic AI amplifies their existing processes — which means it amplifies gaps and inconsistencies as well as strengths. The recommended approach for less mature teams is to sequence the investment correctly: begin with AI-assisted test generation in a bounded context, demonstrate value, and then expand [7].

Tooling selection for agentic QA should be guided by several key criteria beyond marketing claims and feature lists. Integration with existing CI/CD pipelines is non-negotiable: tools requiring parallel infrastructure or manual orchestration defeat the purpose of autonomous testing [2]. Transparency of reasoning is increasingly important: teams need to understand why an AI agent made a particular decision in order to validate it and learn from it over time. Support for human feedback loops is essential: the best agentic QA tools allow teams to correct AI outputs and incorporate those corrections into future behavior, so the tool improves in the specific context of each organization's codebase. Finally, teams should evaluate the vendor's approach to hallucination mitigation and false-positive management — asking not just what the rate is but what mechanisms exist for detection, reporting, and continuous improvement [14].

Defining the role of AI agents clearly within the team's workflow is a governance step that many organizations overlook. Treating AI agents as collaborators — assigning them specific responsibilities, defining the scope of their autonomous authority, and establishing clear feedback loops for human review — produces better results than either accepting AI outputs uncritically or always verifying them in full before acting [7]. The collaborative model recognizes that AI agents have genuine capabilities that justify substantial trust in specific domains, while also recognizing that human judgment must remain engaged in strategic and contextual decisions that exceed the AI's reliable performance range.

Continuous validation of AI-generated test coverage against actual risk areas is a best practice that deserves emphasis because it addresses a subtle failure mode of agentic QA adoption. AI systems that generate tests autonomously can create an illusion of comprehensive coverage — impressive dashboards showing high code-coverage percentages — while missing the specific scenarios that matter most for the organization's users. Defect history analysis, production incident review, and business impact assessment should be regularly fed back into the agentic system's risk modeling to ensure that its coverage priorities remain aligned with real-world quality risks [7].

10 The Future of Quality Assurance — Human-AI Teaming

Looking forward, the trajectory of AI-driven testing and code review points toward a model the industry has come to call human-AI teaming. This is neither the dystopian scenario of AI eliminating QA professionals nor the conservative scenario of AI remaining a marginal supplement to human-dominated processes. It is a genuine partnership in which AI and human capabilities are integrated at a deep level, each performing the tasks for which it is best suited, with the integration managed by organizational structures and governance frameworks that ensure accountability and continuous improvement. This vision is grounded not in optimism but in the practical realities of what AI systems can and cannot reliably do, and what human professionals cannot do at scale and speed [@perfecto2025agenticqa; @allis2025saasriseqa].

The capabilities of agentic QA systems will continue to expand. Test generation from requirements will become more sophisticated, incorporating deeper understanding of domain context and business logic. Self-healing automation will become more reliable, reducing the residual cases where human intervention is required. AI code review will develop better calibration, reducing false positives while improving detection of subtle logical errors and security vulnerabilities. Root-cause analysis will become more accurate and specific, reducing the diagnostic burden on developers [@zyrix2025revolution; @testgrid2025agenticqa]. All of these improvements will extend the boundary of what AI agents can do reliably and autonomously, progressively reducing the scope of tasks that require human judgment for individual-case decisions.

At the same time, the strategic and contextual dimensions of quality assurance will remain central and will in many ways become more important as AI handles the operational burden. The questions of what risks matter most for a particular business, which quality trade-offs are acceptable given resource constraints, how to interpret AI-generated quality data in the context of organizational priorities, and how to design governance frameworks that ensure accountable AI behavior — these define strategic QA leadership, and they are not questions that AI systems are positioned to answer autonomously [10].

Advanced workforce training standards are already emerging in response to the recognition that the QA profession needs new competencies for the agentic era. These standards address not just technical skills — prompt engineering, AI output validation, pipeline configuration — but also analytical and governance skills: how to evaluate AI tool selection decisions, how to design human-AI handoff protocols, how to interpret coverage and quality metrics in an AI-augmented environment, and how to communicate AI-generated quality assessments to non-technical stakeholders [17]. Organizations investing in these training programs now are building organizational capabilities that will compound in value as agentic AI becomes more deeply embedded in their software development processes.

The societal and regulatory dimensions of agentic QA will also demand increasing attention. As AI systems take on greater responsibility for quality decisions in software affecting health, safety, finance, and critical infrastructure, the standards to which those systems are held — and the documentation required to demonstrate compliance — will necessarily evolve. Organizations that develop robust governance frameworks for their AI quality systems now will be better positioned to demonstrate compliance as regulatory requirements crystallize [18].

The agentic era of quality assurance is not a future state to be anticipated but a present reality to be navigated. The tools are here, adoption is accelerating, and the

organizations that engage seriously with both the capabilities and the challenges of agentic AI will define the quality standards the industry considers normal in three to five years. The fundamental insight guiding this navigation: AI and human QA capabilities are not competing for the same resources — they are complementary contributions to a shared goal. When AI handles scale and speed, and humans provide judgment and accountability, the result is not just faster QA but genuinely better QA — which is, after all, what the discipline has always been for.

11 Hebrew Summary / סיכום בעברית

English introduction: This chapter provides a concise bilingual summary of the article's core findings, presented for international and Hebrew-speaking readers.

סיכום: משנה את יסודות בקרת האיכות בעידן הנוכחי, בינה מלאכותית אגנטית (Agentic AI) יכולות כעת לייצר בדיקות אוטומטיות, לתקן תסריטי בדיקה שנשברו, AI בפיתוח תוכנה. מערכות הכל באופן עצמאי וללא התערבות אנושית מתמדת — ולסקור קוד. **However**, human oversight and strategic judgment remain irreplaceable: AI provides speed and scale, while human professionals provide accountability, contextual reasoning, and ethical governance. אתגרי של מודלי שפה גדולים, ניהול יחס אות לרעש (signal-to-noise ratio), המפתח כוללים הזיות (hallucinations) וקביעת מסגרות ממשל ברורות. **The conclusion is clear:** the organizations that invest simultaneously in agentic AI tools and in human capability development will define the quality standards of the next decade. עם שיפוט אנושי ה-AI השילוב של יכולות אחראי הוא המפתח לתוכנה איכותית, בטוחה ומהימנה.

12 Conclusion

The transformation of quality assurance by agentic AI represents one of the most significant shifts in software engineering practice since the introduction of automated testing frameworks. This article has traced that transformation across its key dimensions: the rise of agentic AI as a distinct capability class, the autonomous test generation and self-healing execution capabilities already in production use, the evolution of AI-driven code review from simple static analysis to agentic workflow integration, the genuine challenges of hallucination, false positives, and signal-to-noise management, the essential role of human oversight, the best practices that distinguish effective from ineffective implementations, and the human-AI teaming model that defines the future of the discipline.

Several themes emerge consistently across these dimensions. First, the pressure on QA from AI-accelerated development is real and growing, and the response must be equally ambitious — incremental improvements to traditional QA models are insufficient [19]. Second, agentic AI offers genuine and substantial capabilities, but those capabilities come with genuine risks that require active management rather than optimistic dismissal [@iacis2025perceptions; @diffray2025hallucinations]. Third, human judgment, oversight, and strategic intelligence are not temporary necessities to be engineered away as AI improves — they are permanent features of responsible quality assurance in any era [@mot2025rethinkingqa; @augment2025copilotreview]. Fourth, the organizations that succeed in the agentic QA era will be those that invest simultaneously in technology and human capability, building governance frameworks that enable trustworthy AI autonomy while preserving the accountability that professional quality assurance demands.

The combination of agentic test generation, self-healing automation, AI code review,

and intelligent pipeline integration creates a QA capability that is faster, more comprehensive, and more consistent than human teams operating alone could achieve at scale. The addition of thoughtful human oversight, strategic governance, continuous validation against real-world risks, and ongoing professional development creates a QA organization that can be trusted — by engineering teams, by business stakeholders, and by the users who depend on the software they produce. That combination is the gold standard toward which the quality assurance profession is now moving, and the agentic era is the context in which it will be achieved.

13 Data Visualizations

Further charts generated for this article.

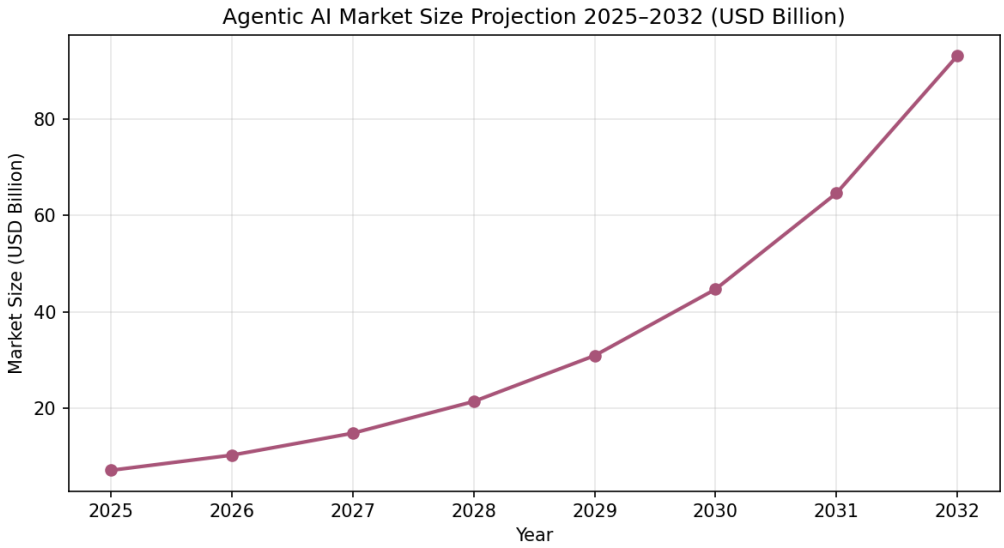

Figure 2: Agentic AI Market Size Projection 2025–2032 (USD Billion)

14 Block Diagram

The diagram below shows a process described in this article.

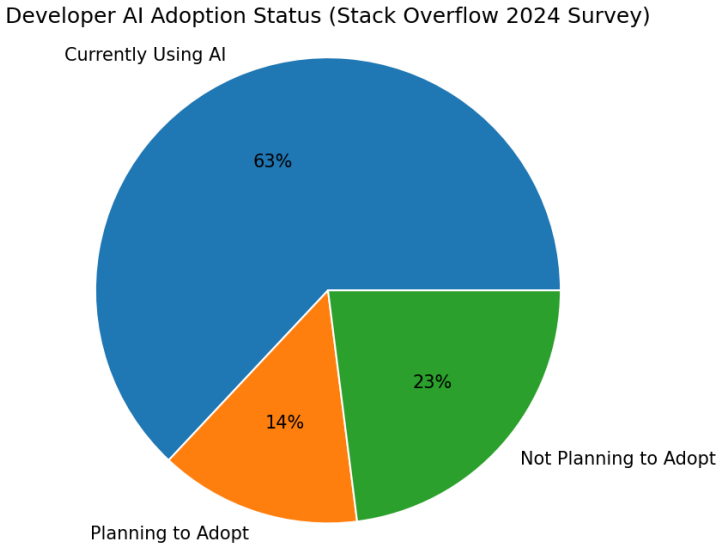

Figure 3: Developer AI Adoption Status (Stack Overflow 2024 Survey)

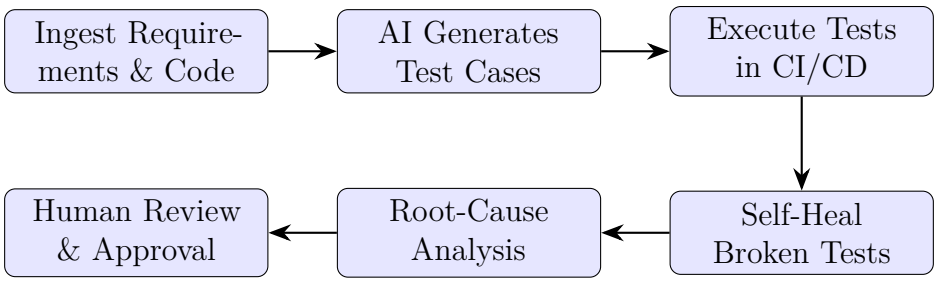

Figure 4: Agentic QA Pipeline: From Requirements to Continuous Delivery

References

[1] Agentic ai in qa: Self-healing tests, root-cause detection & beyond. https://medium.com/@AziroTech/agentic-ai-in-qa-self-healing-tests-root-cause-detection-beyond-0978983f7bf1, 2026. Accessed 2026.

[2] Agentic ai in test automation: Self-healing tests that learn. https://pie.inc/blog/agentic-ai-test-automation/, 2026. Accessed 2026.

[3] Agentic ai is revolutionizing test automation in 2025. https://zyrix.ai/blogs/agentic-ai-revolutionizing-test-automation-2025/, 2026. Accessed 2026.

[4] Agentic ai market report 2025–2032. https://www.marketsandmarkets.com/Market-Reports/agentic-ai-market-208190735.html, 2026. Accessed 2026.

[5] Agentic ai testing: The future of autonomous software qa. https://testgrid.io/blog/agentic-ai-testing/, 2026. Accessed 2026.

[6] Agentic ai: The shift to autonomous software testing. https://testquality.com/agentic-ai-the-shift-to-autonomous-software-testing/, 2026. Accessed 2026.

[7] Agentic quality assurance: A guide to ai-driven qa. https://www.tricentis.com/learn/agentic-quality-assurance, 2026. Accessed 2026.

[8] Ai code review automation: Complete guide 2025. https://www.digitalapplied.com/blog/ai-code-review-automation-guide-2025, 2026. Accessed 2026.

[9] Ai copilot code quality: 2025 data suggests 4x growth. https://www.gitclear.com/ai_assistant_code_quality_2025_research, 2026. Accessed 2026.

[10] Ai testing in 2026: Why signal, trust, and intentional choices matter. https://applitools.com/blog/ai-testing-strategy-in-2026/, 2026. Accessed 2026.

[11] Future-proofing qa teams: Preparing for the agentic ai testing era. https://www.virtuosoqa.com/post/future-proofing-qa-teams-preparing-for-the-agentic-ai-testing-era-virtuosoqa, 2026. Accessed 2026.

[12] Github ai code review: 8 copilot pr automation features. https://www.augmentcode.com/tools/github-copilot-ai-code-review, 2026. Accessed 2026.

[13] How agentic ai is redefining software testing. https://www.virtuosoqa.com/post/agentic-ai-testing-revolution, 2026. Accessed 2026.

[14] Llm hallucinations in ai code review. https://diffray.ai/blog/llm-hallucinations-code-review/, 2026. Accessed 2026.

[15] Managing software qa while using agentic engineering and ai coding. https://www.saasrise.com/blog/managing-software-quality-assurance-qa-while-using-agentic-engineering-and-ai-coding, 2026. Accessed 2026.

[16] Perceptions and challenges of ai-driven code reviews. https://iacis.org/iis/2025/2_iis_2025_346-360.pdf, 2026. Accessed 2026.

[17] Qa trends for 2026: Ai, agents, and the future of testing. https://www.tricentis.com/blog/qa-trends-ai-agentic-testing, 2026. Accessed 2026.

[18] Rethinking qa strategy with next-gen ai & agentic ai. https://club.ministryoftesting.com/t/rethinking-qa-strategy-with-next-gen-ai-agentic-ai/86604, 2026. Accessed 2026.

[19] Testing in the age of agentic ai. https://atos.net/en/blog/testing-in-the-age-of-agentic-ai, 2026. Accessed 2026.

[20] Toward agentic code review: Reimagining the process in the ai era. https://conf.researchr.org/details/icse-2026/jaws-2026-papers, 2026. Accessed 2026.